Тестове завдання.
Data Engineer

Задача 1 Вигрузка з партнерського API

Робочий каталог: `task1_api/`

Є партнер, який віддає щоденний звіт по трафіку й виручці. Треба забирати ці дані кожні 15 хвилин і складати в таблицю `partner_report` так, щоб аналітик міг просто взяти її і рахувати, не перепитуючи, чи там усе на місці.

Процес має жити сам. Підняли `docker compose up` і забули. Ніхто не смикатиме руками `make extract` кожні п'ятнадцять хвилин. Як ви це зробите, нам не принципово: цикл в entrypoint, cron усередині контейнера, окремий сервіс. Напишіть тільки, чому обрали саме такий варіант.

Що лежить у каталозі

`openapi_public.yaml`	документація API, як її публікує партнер
`mock_api/`	сам сервер. Заглядати в код можна, просто скажіть, що заглядали
`extractor/main.py`	ваш код. Поки що там скелет із `NotImplementedError`
`warehouse/init.sql`	цільова таблиця й таблиця стану. Схему міняти можна, напишіть навіщо

```
make api-up       # API на :8080, Postgres на :5432
make extract      # запускає ваш код
make psql         # подивитись, що вийшло
```

Що має вміти результат

Повнота	жоден рядок не загубився і не задублювався
Ідемпотентність	запустили двічі поспіль, стан таблиці однаковий
Стійкість	впало посеред вигрузки, таблиця не лишилась у напівстані
Ввічливість	не впирається в ліміти партнера і не ретраїть наосліп
Коректність	цифри в таблиці збігаються з тим, що віддав API
Логи	видно, що зробив запуск: яке вікно, скільки рядків, скільки запитів, що змінилось
Відновлення	після кількагодинного простою сам доганяє пропущене, без дірки в даних

З цього списку найбільше нас цікавить рядок про цифри. Дуже легко зробити процес, який бадьоро возить рядки в базу, і не помітити, що половина з них неправильні. Перевірте.

Що здати: `FINDINGS.txt`

Окремий файл про те, де API поводиться не так, як обіцяє документація. По кожному випадку: що написано в специфікації, що насправді робить сервер, як ви на це натрапили і чим це загрожує, якщо документації повірити.

Скажемо чесно: без цього файлу ми не побачимо різниці між людиною, яка розібралась в API, і людиною, якій пощастило.

Задача 2 Переведення Scheduled Queries у Dataform

Робочий каталог: `task2_estate/`

За чотири роки навколо аналітики наросло 129 BigQuery Scheduled Queries. Писали їх чотири різні людини в різний час, ніхто ні з ким не домовлявся, інструменту, який стежив би за залежностями, не було. Двох із тих людей у компанії вже немає, спитати нема в кого.

Так, це безлад. Ми його не приховуємо, бо саме з ним і доведеться працювати.

Переїжджаємо на Dataform. Хочемо не просто перекласти ті самі запити в git. Хочемо, щоб залежності стало видно, щоб однакові перетворення перестали жити в п'ятнадцяти копіях і щоб можна було нарешті сказати, скільки все це коштує.

Переписувати всі 129 не треба. Треба зрозуміти, що з цього взагалі варто тягнути далі, і показати один маршрут, зроблений як слід.

> Запитів свідомо більше, ніж реально прочитати за адекватний час. Не читайте їх руками, напишіть щось, що їх розбере. Увагу краще витратити на рішення.

Що лежить у каталозі

Файл	Що це
`scheduled_queries/*.sql`	129 файлів, назва `<config_id>__<dataset>__<table>.sql`
`transfer_configs.json`	вивід `bq ls/show --transfer_config`: розклад, тайм-зона, `write_disposition`, призначення, власник, стан
`run_history.csv`	історія запусків, 90 днів для щоденних і 14 для щогодинних: `start_time`, `end_time`, `state`, `error_message`, `total_bytes_processed`, `rows_written`
`table_catalog.csv`	зріз `INFORMATION_SCHEMA`: таблиці, розміри, партиціювання, `last_modified`
`downstream_usage.csv`	хто читав кожну таблицю за 30 днів: інші запити, BI-дашборди, ad-hoc
`schemas.json`	колонки й типи вихідних `raw.*` таблиць
`dataform/`	порожній скелет цільового проєкту з `workflow_settings.yaml`

Три CSV раджу відкрити на початку. Частину проблем у самому SQL не видно взагалі, вони вилазять тільки з історії запусків і з того, хто які таблиці читає.

Ціни BigQuery: on-demand приблизно 6.25 USD за TiB відсканованого, мінімум 10 MB на запит.

`AUDIT.txt` і ваш інструмент у `tools/`

- Граф залежностей, зібраний автоматично з SQL. Скрипт покладіть у `tools/`, ми його читатимемо разом з рештою.
- Реєстр проблем: що не так, з посиланням на `config_id` і на доказ із наданих файлів. Якщо доказу немає, це поки що здогадка, так і напишіть.
- Список того, що варто вимкнути замість переносити. Отут докази потрібні найсильніші. Вимкнути живий пайплайн дорого, і ми не хочемо дізнатись про це від фінансистів.

`COST.txt`

Скільки цей склад запитів коштує за добу, куди саме йдуть гроші і що дасть найбільшу економію за найменшої зміни. Рахуйте з `run_history.csv`. Окремо скажіть, що можна полагодити просто зараз, а що впирається в рішення власника даних.

`PLAN.txt`

- Шари, неймінг, теґи. Що стає спільним макросом в `includes`, що декларацією джерела.
- Порядок міграції і чому саме такий.
- Секція `Equivalence`: як довести, що перенесений маршрут дає той самий результат. Складу даних у вас немає, тому напишіть план перевірки. Які саме запити ви б запустили і що очікуєте побачити. Цю секцію ми читаємо уважніше за інші.
- Що ви вирішили не переносити і чому.
- Як ви робили б решту: де тут механічна робота, яку можна віддати скрипту чи асистенту, а де треба думати над кожним файлом окремо.

Робочий Dataform-код одного маршруту

Перенесіть маршрут, який живить `reports.revenue_daily`, від `raw.*` до споживачів. Хто саме споживачі, покаже `downstream_usage.csv`. Вийде приблизно 12–18 actions .

- `make compile` проходить без помилок.
- Логіка повторює оригінал. Якщо свідомо щось міняєте, запишіть це в `DECISIONS.txt` разом з причиною.
- Типи actions (`table`, `incremental`, `view`, `assertion`) обирайте під поведінку оригіналу, а не за замовчуванням.

`DECISIONS.txt`

Рішення, які ви ухвалили самі. І окремо не більше п’яти питань , які ви поставили б власнику даних перед тим, як переносити решту.

Питання мають бути такі, що від відповіді залежить код. «Чи все правильно?» таким питанням не є. А от «у цих чотирьох запитах Туреччина потрапляє в інший регіон, ніж у решті, яка версія правильна» — це воно.

Як здати

Git-репозиторій, приватний GitHub або GitLab. Або архів разом із `.git`.

```
FINDINGS.txt                задача 1, API проти документації
task1_api/extractor/        задача 1, ваш код
AUDIT.txt                   задача 2, карта складу й реєстр проблем
COST.txt                    задача 2, вартість і де економія
PLAN.txt                    задача 2, цільовий дизайн і Equivalence
DECISIONS.txt               задача 2, рішення й питання
task2_estate/dataform/      задача 2, перенесений маршрут
tools/                      задача 2, ваш інструмент розбору
```

У корені покладіть короткий `README.txt`: як запустити екстрактор, як запустити ваш інструмент розбору, як скомпілювати Dataform-проєкт.

Додаток. Як запустити середовище

Тільки про запуск середовища. Самі задачі описані окремо.

Усе працює в Docker. Хмарних доступів не треба, на рантаймі мережа не потрібна. Перша збірка образів мережу використовує, бо тягне base images.

```
make build          # зібрати образи, перший раз кілька хвилин
make help           # усі таргети з описом
```

Задача 1. Партнерське API

```
make api-up         # mock-api на :8080, Postgres на :5432
curl -s localhost:8080/healthz
make extract        # запускає task1_api/extractor/main.py
make psql           # подивитись, що ви завантажили
```

Креденшели пісочниці лежать у `docker-compose.yml`. Вони несправжні. Читати `task1_api/mock_api/` можна, тільки напишіть про це у `FINDINGS.md`.

Задача 2. Склад запитів

```
make estate-stats   # перевіряє, що файли складу читаються
make shell          # python 3 + sqlglot + networkx + pandas + duckdb + jq
make compile        # компілює task2_estate/dataform, має вийти з кодом 0
```

Складу даних для задачі 2 немає. `raw.*` не існує ніде, куди можна надіслати запит. Жоден зі 129 запитів запустити не вийде, і `dataform run` не спрацює. Працює тільки `dataform compile`. Колонки й типи вихідних таблиць описані лише в `schemas.json`.

Це обмеження вправи, а не недогляд. Що з цим робити, написано в завданні, у частині про `PLAN.md` і секцію `Equivalence`.

Що є в образі лаби

Інструмент	Навіщо
`dataform` CLI 3.x	єдиний спосіб перевірити ваші actions без складу даних
`sqlglot`	нормально парсити BigQuery SQL, знадобиться для графа залежностей
`networkx`	алгоритми на графах, якщо захочете
`pandas`, `duckdb`	запитувати CSV як таблиці
`jq`	для `transfer_configs.json`

Нічого з цього не обов'язкове, користуйтесь своїм інструментарієм, якщо так зручніше. Фіксовані лише дві речі: `make compile` має виходити з кодом 0, а ваш інструментарій має лежати в репозиторії.